

4-LAB

Task-1. Retrieve all airline names in uppercase.

SQL-Code:

The screenshot shows the PostgreSQL IDE interface. On the left, the Object Explorer displays the database structure, including the 'airline' table under the 'public' schema. The main query editor contains the following SQL code:

```
1 select upper(airline_name) from airline
2
```

The Data Output pane at the bottom shows the results of the query, displaying 15 rows of uppercase airline names:

upper
KAZAIR
AIREASY
FLYHIGH
FLYFLY
GLOBALAIRWA...
FLY6
GLOBALAIR7
AIR8
SKYAIR9
FLY10
GLOBALAIR11
AIR12
SKYAIR13
FLY14
GLOBALAIR15

The status bar at the bottom indicates 'Total rows: 150' and 'Query complete 00:00:00.053'.

Task-2. Replace any occurrence of the word "Air" in airline names with "Aero".

SQL-Code:

The screenshot shows the PostgreSQL IDE interface. The main query editor contains the following SQL code:

```
1 select replace(airline_name, 'Air', 'Aero')
2 as airline_name
3 from airline
4
```

The Data Output pane at the bottom shows the results of the query, displaying 15 rows of airline names with 'Air' replaced by 'Aero':

airline_name
KazAero
AeroEasy
FlyHigh
FlyFly
GlobalAeroWa...
Fly6
GlobalAero7
Aero8
SkyAero9
Fly10
GlobalAero11
Aero12
SkyAero13
Fly14
GlobalAero15

A green status bar at the bottom indicates 'Successfully run. Total query runtime: 55 msec. 150 rows affected.' The status bar also shows 'Total rows: 150' and 'Query complete 00:00:00.055'.

Task-3. Find all flight numbers that coordinates with both airline 1 and airline 2.

SQL-Code:

The screenshot shows the PostgreSQL IDE interface. The Object Explorer on the left displays the database structure, including the 'airline' table. The main query window contains the following SQL code:

```
1 select flight_id from flights
2 where airline_id = 1 or airline_id = 2;
3
```

The Data Output pane shows the results of the query, displaying 61 rows of flight IDs. The status bar at the bottom indicates 'Total rows: 61' and 'Query complete 00:00:00.087'.

flight_id [PK] integer
5
9
11
12
14
22
24
26
32
44
48
53
65
66
70
71
75
78
81
84
87
89
91
93
95

Task-4. Retrieve airports that contain the word "Reginal" and "Air" in their names.

SQL-Code:

The screenshot shows the PostgreSQL IDE interface. The Object Explorer on the left displays the database structure, including the 'airport' table. The main query window contains the following SQL code:

```
1 select * from airport
2 where airport_name like '%Reginal%' and airport_name like '%Air%';
3
```

The Data Output pane shows the results of the query, displaying 3 rows of airport information. The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.071'. A green message box at the bottom right states 'Successfully run. Total query runtime: 71 msec. 3 rows affected.'

airport_id [PK] integer	airport_name character varying (50)	state character varying (50)	city character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	Reginal Air Airport	Akmla	Los Angeles	2026-09-27 20:45:52.715643	2026-09-27 20:45:52.715643
2	Reginal Air Airport	England	Paris	2026-09-27 20:45:52.715643	2026-09-27 20:45:52.715643
3	Reginal Air Airport	Ile de France	Paris	2026-09-27 20:45:52.715643	2026-09-27 20:45:52.715643

Task-5. Retrieve passenger names and format their birth dates as 'Month DD,YYYY'.

SQL-Code:

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, with the 'passengers' table selected under the 'airport' database. The main query editor displays the following SQL code:

```
1 select first_name,  
2 to_char(date_of_birth, 'Month DD, YYYY') as birth_date  
3 from passengers  
4
```

The 'Data Output' pane shows the results of the query, displaying 22 rows of passenger data. The columns are 'first_name' and 'birth_date'. The data is as follows:

first_name	birth_date
Passenger1	January 22, 2013
Passenger2	August 16, 1980
Passenger3	February 18, 1989
Passenger4	October 01, 1996
Passenger5	November 25, 20...
Passenger6	November 20, 19...
Passenger7	January 06, 2020
Passenger8	October 24, 2014
Passenger9	January 06, 2018
Passenger10	October 02, 1984
Passenger11	September 09, 20...
Passenger12	September 05, 19...
Passenger13	July 22, 2005
Passenger14	July 10, 2012
Passenger15	December 31, 20...
Passenger16	March 08, 1995
Passenger17	November 03, 19...
Passenger18	September 09, 20...
Passenger19	July 20, 2010
Passenger20	May 05, 1981
Passenger21	December 11, 20...
Passenger22	May 30, 1988

The status bar at the bottom indicates 'Total rows: 200' and 'Query complete 00:00:00.074'.

Task-6. Find flight numbers that have been delayed based on the actual arrival time.

SQL-Code:

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, with the 'flights' table selected under the 'airport' database. The main query editor displays the following SQL code:

```
1 select flight_id from flights  
2 where act_arrival_time > sch_arrival_time;  
3
```

The 'Data Output' pane shows the results of the query, displaying 127 rows of flight data. The column is 'flight_id'. The data is as follows:

flight_id
4
5
7
8
9
11
12
14
17
18
22
24
26
27
28
32
33
34
35
36
37
38
39
41
44
46
48
49
52
53

The status bar at the bottom indicates 'Total rows: 127' and 'Query complete 00:00:00.081'.

Task-7. Find flights that arrived late according to their actual arrival time compared to the scheduled arrival time.

SQL-Code:

The screenshot shows a SQL query in the 'Query' pane of SQL Studio. The query is:

```
1 select flight_id, act_arrival_time - sch_arrival_time as delay_time
2 from flights
3 where act_arrival_time > sch_arrival_time;
4
```

The 'Data Output' pane shows the results of the query, displaying columns 'flight_id' and 'delay_time'. The results show 127 rows of flights that arrived late.

flight_id	delay_time
4	00:10:00
5	00:10:00
7	00:10:00
8	00:10:00
9	00:10:00
11	00:10:00
12	00:10:00
14	00:10:00
17	00:10:00
18	00:10:00
22	00:10:00
24	00:10:00
26	00:10:00
27	00:10:00
28	00:10:00
32	00:10:00
33	00:10:00
34	00:10:00
35	00:10:00
36	00:10:00
37	00:10:00
38	00:10:00
39	00:10:00
41	00:10:00
44	00:10:00

Total rows: 127 Query complete 00:00:00.125

Task-8. Show airlines from any of: ('France','Portugal','Poland') created between '2023-11-01' and '2024-03-31'.

SQL-Code:

The screenshot shows a SQL query in the 'Query' pane of SQL Studio. The query is:

```
1 select * from airline
2 where airline_country in ('FRANCE', 'PORTUGAL', 'POLAND')
3 and created_at between '2023-11-01' and '2024-03-31';
4
```

The 'Data Output' pane shows the results of the query, displaying columns 'airline_id', 'airline_code', 'airline_name', 'airline_country', 'created_at', 'updated_at', and 'info'. The results show 5 rows of airlines.

airline_id	airline_code	airline_name	airline_country	created_at	updated_at	info
7	A7	GlobalAir7	FRANCE	2024-01-24 14:35:27.473053	2026-10-04 20:56:37.75748	[null]
12	A12	Air12	POLAND	2023-11-19 14:09:17.135912	2026-10-04 20:56:37.75748	[null]
14	A14	Fly14	POLAND	2024-03-21 14:57:21.666625	2026-10-04 20:56:37.75748	[null]
43	A43	GlobalAir43	FRANCE	2024-01-23 20:09:51.502555	2026-10-04 20:56:37.75748	[null]
95	A95	GlobalAir95	PORTUGAL	2023-11-01 15:40:44.004681	2026-10-04 20:56:37.75748	[null]

Total rows: 5 Query complete 00:00:00.066

Successfully run. Total query runtime: 66 msec. 5 rows affected.

Task-9. Find top3 overweighted baggage with more than 25kg.

SQL-Code:

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, including tables like 'airline', 'airport', and 'baggage'. The 'baggage' table is selected, showing its columns: 'baggage_id' (integer), 'weight_in_kg' (numeric), 'created_at' (timestamp), 'updated_at' (timestamp), and 'booking_id' (integer). The main query editor displays the following SQL code:

```
1 SELECT * FROM baggage
2 where weight_in_kg > 25
3 order by weight_in_kg desc
4
5 limit 3
```

The 'Data Output' pane shows the results of the query, displaying 3 rows of baggage data:

baggage_id (PK) integer	weight_in_kg numeric (4,2)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer
105	29.97	2026-09-27 21:14:48.235034	2026-09-27 21:14:48.235034	105
179	29.89	2026-09-27 21:14:48.235034	2026-09-27 21:14:48.235034	179
168	29.80	2026-09-27 21:14:48.235034	2026-09-27 21:14:48.235034	168

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.133'. A green notification box at the bottom right states 'Successfully run. Total query runtime: 133 msec. 3 rows affected.'

Task-10. Find the youngest passengers' full name.

SQL-Code:

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, including tables like 'airline', 'airport', and 'baggage'. The 'baggage' table is selected, showing its columns: 'baggage_id' (integer), 'weight_in_kg' (numeric), 'created_at' (timestamp), 'updated_at' (timestamp), and 'booking_id' (integer). The main query editor displays the following SQL code:

```
1 select first_name || ' ' || last_name as full_name
2 from passengers
3 order by date_of_birth desc
4
5 limit 1
```

The 'Data Output' pane shows the results of the query, displaying 1 row of passenger data:

full_name text
Passenger611 Lastname61

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.072'. The bottom right corner shows 'LF Ln 4, Col 8'.

Task-11. Find the cheapest booking price on each booking platform.

SQL-Code:

The screenshot shows the PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, including tables like 'airport', 'baggage', 'booking', and 'booking_flight'. The 'booking' table is selected, showing its columns: 'booking_id', 'flight_id', 'passenger_id', 'booking_platform', 'created_at', 'updated_at', 'status', and 'ticket_price'. The main query editor displays the following SQL query:

```
select booking_platform, min(ticket_price) as cheapest_price
from booking
group by booking_platform
```

The 'Data Output' pane shows the results of the query:

booking_platform	cheapest_price
Agency	10535.15
Mobile App	10967.55
Website	1500.00

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.058'.

Task-12. Return airlines whose airline_code contains a digit.

SQL-Code:

The screenshot shows the PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, including tables like 'airport', 'baggage', 'booking', and 'booking_flight'. The 'airline' table is selected, showing its columns: 'airline_id', 'airline_code', 'airline_name', 'airline_country', 'created_at', 'updated_at', 'info', and 'ticket_price'. The main query editor displays the following SQL query:

```
select * from airline
where airline_code ~ '[0-9]'
```

The 'Data Output' pane shows the results of the query, displaying a list of airlines with their codes containing digits. The status bar at the bottom indicates 'Total rows: 145' and 'Query complete 00:00:00.109'.

Task-13. List the top5 most recently created airlines.

SQL-Code:

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, including tables like 'airline', 'baggage', and 'booking'. The main query editor on the right contains the following SQL code:

```
1 select * from airline
2 order by created_at desc
3 limit 5
```

The 'Data Output' pane shows the results of the query, displaying 5 rows of airline data:

airline_id	airline_code	airline_name	airline_country	created_at	updated_at	info
5	KR	Global Airways	United Kingdom	2026-09-27 22:27:29.530475	2026-09-27 00:00:00	[null]
3	FH	FlyHigh	BRASIL	2026-09-27 20:29:13.600869	2026-09-27 20:29:13.600869	[null]
4	LNK	FlyFly	POLAND	2026-09-27 20:29:13.600869	2026-09-27 20:29:13.600869	[null]
2	AE	AirEasy	FRANCE	2026-09-27 20:29:13.600869	2026-09-27 20:29:13.600869	[null]
1	KAZ	KazAir	TURKEY	2026-09-27 19:05:20.781285	2026-09-27 19:05:20.781285	[null]

A status message at the bottom indicates: 'Successfully run. Total query runtime: 107 msec. 5 rows affected.'

Task-14. Return all rows where booking_id is between 200 and 300 inclusive and check_result <> 'Checked'.

SQL-Code:

The screenshot shows the same PostgreSQL IDE interface. The query editor now contains the following SQL code:

```
1 select * from booking
2 where booking_id between 200 and 300
3 and status <> 'Checked'
```

The 'Data Output' pane shows the results of the query, displaying 2 rows of booking data:

booking_id	flight_id	passenger_id	booking_platform	created_at	updated_at	status	ticket_price
200	73	84	Agency	2026-09-27 21:01:16.78182	2026-09-27 21:01:16.78182	Confirmed	16248.35
300	73	20	Website	2026-09-30 14:34:38.306902	2026-09-30 14:34:38.306902	Confirmed	1500.00

A status message at the bottom indicates: 'Successfully run. Total query runtime: 81 msec. 2 rows affected.'

Task-15. Baggage checks where update_at is in the same month as created_at but occurs earlier than created_at.

SQL-Code:

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, including tables like 'airport', 'baggage', and 'booking'. The 'baggage' table is selected, showing its columns: 'baggage_id' (integer), 'weight_in_kg' (numeric(4,2)), 'created_at' (timestamp without time zone), 'updated_at' (timestamp without time zone), and 'booking_id' (integer). The 'booking' table is also visible, showing columns like 'booking_id' (integer), 'flight_id' (integer), 'passenger_id' (integer), 'booking_platform' (character varying(50)), 'created_at' (timestamp without time zone), 'updated_at' (timestamp without time zone), 'status' (character varying(50)), and 'ticket_price' (numeric(7,2)).

The main query editor displays the following SQL code:

```
1 SELECT *
2 FROM baggage_check
3 WHERE DATE_TRUNC('month', updated_at) = DATE_TRUNC('month', created_at)
4 AND updated_at < created_at;
5
```

The 'Data Output' pane shows the results of the query, which are empty. The status bar at the bottom indicates 'Total rows: 0' and 'Query complete 00:00:00.079'.